

STA 5635 HW 10

Palmer Swanson

March 26, 2026

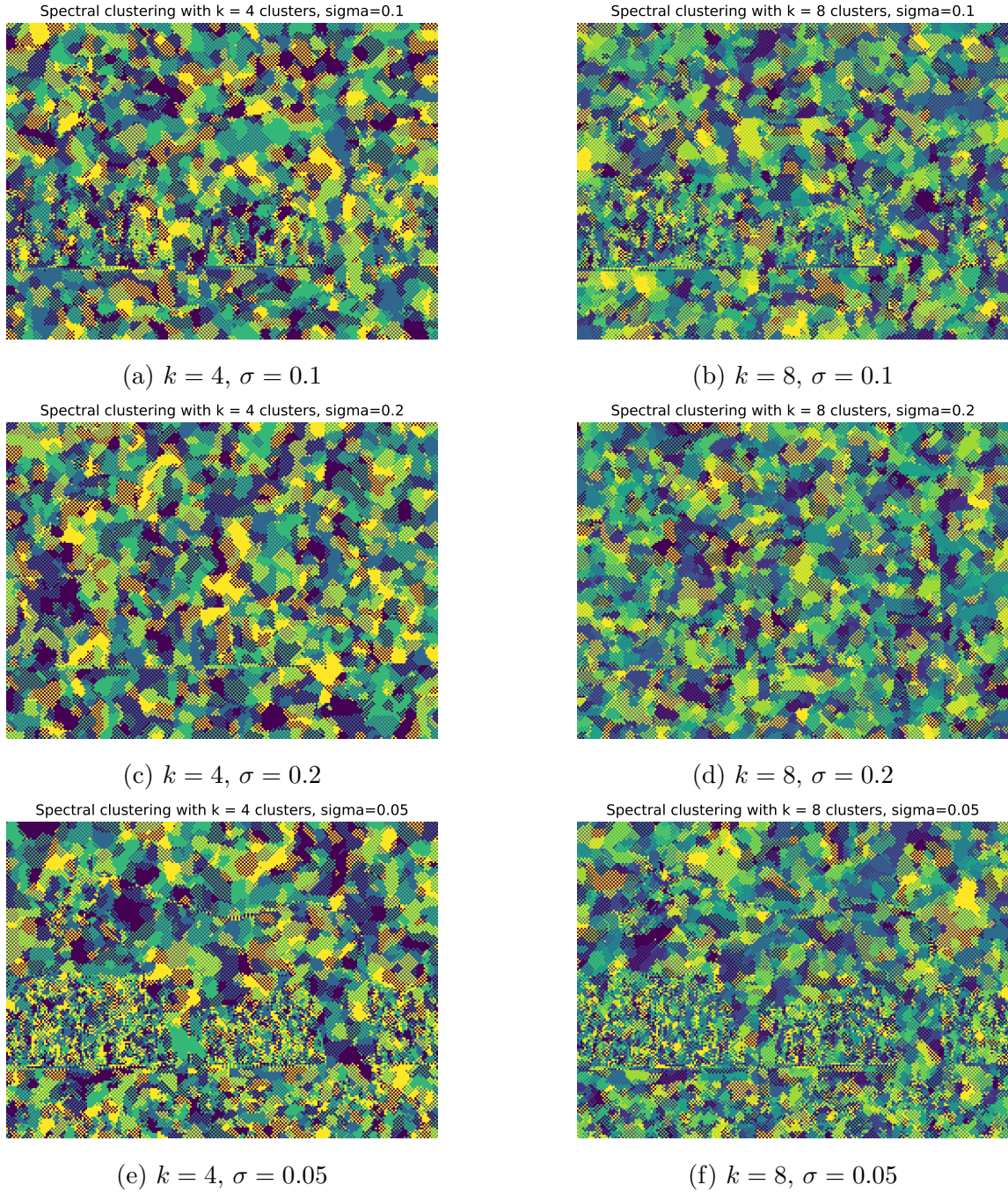


Figure 1: Spectral clustering results for different values of k (clusters) and σ . Part a.



Figure 2: Mean-color images for different values of k (clusters) and σ . Part b

Code

```
1 import numpy as np
2 from scipy.sparse import diags
3 from sklearn.cluster import KMeans
4 import matplotlib.pyplot as plt
5 from PIL import Image
6 from scipy.sparse import coo_matrix
7 import torch
8
9 #found this for working with images
10 #https://pillow.readthedocs.io/en/stable/reference/Image.html
11 #https://stackoverflow.com/questions/51321960/import-image-in-python
12 #https://pillow.readthedocs.io/en/stable/reference/Image.html#PIL.Image.Image.convert
13
14 #load the image and convert to numpy array and normalize
15 img = Image.open('assignments/hw10/data/horse027.jpg').convert("RGB")
16 img_array = np.array(img)
17 img_array = img_array/255.0
18
19 #function for affinity matrix
20 #use sparse matrices for speed
21 #https://www.geeksforgeeks.org/python/how-to-create-a-sparse-matrix-in-python/
22 def build_affinity_matrix(img_array, sigma):
23     #get image dimensions and total number of pixels
24     height, width, channels = img_array.shape
25     n_pixels = height*width
26
27     #initialize lists to build sparse matrix
28     rows = []
29     cols = []
30     data = []
31
32     #loop over each pixel in the image
33     for row in range(height):
34         for column in range(width):
35
36             #get the RGB vector for the current pixel
37             I_i = img_array[row, column]
38
39             #get index of pixel. each row has width elements, and before row r,
there r width elements. then add the column index
40             i = (row*width) + column
41
42             #loop over the neighbors (up, down, left, right) and compute the
affinity. Need to check for boundary conditions
```

```

43         for neighbor_row, neighbor_column in [(row-1, column), (row+1,
44         column), (row, column-1), (row, column+1)]:
45
46             #check if the neighbor is within the image boundaries
47             if 0 <= neighbor_row < height and 0 <= neighbor_column < width:
48
49                 #get RGB vector for neighbor
50                 I_j = img_array[neighbor_row, neighbor_column]
51
52                 #get neighbor index
53                 j = (neighbor_row*width) + neighbor_column
54
55                 #compute the squared euclidian distance
56                 diff = I_i - I_j
57                 dist_sq = np.sum(diff**2)
58
59                 #compute the affinity and store in the lists for the sparse
60                 matrix
61                 affin = np.exp(-dist_sq / sigma**2)
62                 rows.append(i)
63                 cols.append(j)
64                 data.append(affin)
65
66             #build the sparse affinity matrix
67             A = coo_matrix((data, (rows, cols)), shape=(n_pixels, n_pixels))
68
69             #https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.coo_matrix.tocsr.html
70             return A.tocsr()
71
72 #function for spectral clustering
73 #build A on CPU. Run svd_lowrank on torch
74 #lot of debugging stuff in here. Had a hard time
75 def spectral_clustering(image_array, sigma, n_clusters, random_state):
76
77     #image dimensions and total number of pixels
78     height, width, channels = image_array.shape
79
80     #set the torch device and random seed
81     if torch.cuda.is_available():
82         device = torch.device('cuda')
83     else:
84         device = torch.device('cpu')
85
86     #build the affinity matrix
87     A = build_affinity_matrix(image_array, sigma)
88     #print("build A")

```

```

87
88 #need the Degree matrix. Simply the sum of the four neighbors for each pixel
89 #need sparse matrix
90 #https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.diags.html
91 #flatten turns this into a simple 1D array
92 D = np.array(A.sum(axis=1)).flatten()
93
94 #need D inverse square root
95 D_inv_sqrt = diags(D**(-0.5))
96
97 #feed this into svd. Pg 27 in notes
98 L = D_inv_sqrt @ A @ D_inv_sqrt
99
100 #convert to COO format for torch
101 L_coo = L.tocoo()
102
103 indices = np.vstack((L_coo.row, L_coo.col))
104 indices = torch.from_numpy(indices).long().to(device)
105 values = torch.from_numpy(L_coo.data).float().to(device)
106
107 L_tensor = torch.sparse_coo_tensor(
108     indices,
109     values,
110     size=L.shape,
111     device=device
112 )
113
114 #run svd_lowrank on torch
115
116 #https://pytorch.org/docs/stable/generated/torch.linalg.svd.html#torch.linalg.svd
117 #q needs to be a "slightlyt overestimates rank of the matrix"
118 #increaing niter can lead to better approximations
119 U, S, Vh = torch.svd_lowrank(L_tensor, q=n_clusters+5, niter=10)
120 #print("svd_lowrank")
121
122 #take first n_clusters columns of U
123 #each row of E is the embedding of a pixel in the n_clusters dimensional
124 space. We will run k-means on these
125 E = U[:, :n_clusters].cpu().numpy()
126
127 #normalize the rows of E. Need to avoid dividing by zero. Ran into issues
128 here if we didn't
129 E_row_norms = np.linalg.norm(E, axis=1, keepdims=True)
130 E_row_norms = np.maximum(E_row_norms, 1e-12)
131 E_normalized = E / E_row_norms
132
133 #k-means

```

```

131 kmeans = KMeans(n_clusters=n_clusters, random_state=random_state, n_init=10)
132 labels = kmeans.fit_predict(E_normalized)
133 #print("kmeans")
134
135 #reshape the labels back to the image dimensions for visualization
136 label_image = labels.reshape(height, width)
137
138 return label_image, labels
139
140
141 #actual problem
142 sigmas = [0.1, 0.2, 0.05]
143 clusters = [4,8]
144
145 #loop over the various sigma values
146 for sigma in sigmas:
147
148     #loop over the clusters
149     for cluster in clusters:
150
151         #image dimensions and total number of pixels
152         height, width, channels = img_array.shape
153
154         #run clustering
155         label_image, labels = spectral_clustering(
156             #image_array=img_array,
157             image_array=img_array,
158             sigma=sigma,
159             n_clusters=cluster, random_state=123
160         )
161
162         #plotting part a, clustering results
163         #https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.imshow.html
164         plt.figure()
165         plt.imshow(label_image)
166         plt.title(f"Spectral_clustering_with_k={cluster}_clusters,_"
167 sigma={sigma}")
168         plt.axis("off")
169         #plt.show()
170
171         plt.savefig(f"assignments/hw10/figures/clusters_k{cluster}_sigma{sigma}.pdf",
172 dpi=400, bbox_inches="tight")
173         plt.close()
174
175     #part b
176     #for each cluster, compute the means of all R, G, and B values for the
177     pixels in that cluster

```

```

174     #place that mean at all locations of the pixels in that cluster.
175
176     label_grid = labels.reshape(height, width)
177
178     #don't need to specify dims with zeros_like
179     color_image = np.zeros_like(img_array)
180
181     #loop over each cluster
182     for i in range(cluster):
183         print(i)
184         #find all pixels in cluster i
185         indicator = (label_grid == i)
186
187         #select pixels in the original image that belong to cluster i
188         #shape should be (#pixels in cluster, 3)
189         temp = img_array[indicator]
190
191         #compute the mean R, G, and B values for the pixels in cluster i
192         mean_red = temp[:, 0].mean()
193         mean_green = temp[:, 1].mean()
194         mean_blue = temp[:, 2].mean()
195         mean_color = np.array([mean_red, mean_green, mean_blue])
196
197         #replace all pixels in cluster with the mean color
198         color_image[indicator] = mean_color
199
200     #plot this mean color image
201     plt.figure()
202     plt.imshow(color_image)
203     plt.title(f"Mean_color_image_(k={cluster},_sigma={sigma})")
204     plt.axis("off")
205     #plt.show()
206
207     plt.savefig(f"assignments/hw10/figures/mean_k{cluster}_sigma{sigma}.pdf",
                dpi=400, bbox_inches="tight")
    plt.close()

```